

Zsana Photo - MVP fejlesztési specifikáció v4

0. Dokumentum célja

Ez a dokumentum egy AI coding agent (különösen Claude Code, Cursor Agent, Codex vagy hasonló eszköz) számára készült, közvetlenül implementálható specifikáció.

A cél egy **valóban használható első verzió** elkészítése, amelynek legfontosabb funkciója az online időpontfoglalás.

Az alkalmazás:

- Next.js full-stack alkalmazás legyen;
- egyetlen Git repositoryban tartalmazza a frontend és backend logikát;
- lokálisan és Vercelen is fusson;
- valódi PostgreSQL adatbázist használjon;
- a foglalásokat ténylegesen perzisztálja;
- rendelkezzen admin felülettel;
- a foglalásokról e-mailt küldjön az ügyfélnek és az adminnak;
- támogassa az automatikus és manuális foglalás-jóváhagyást;
- később bővíthető legyen Pixieset, fizetés, számlázás és privát galériák irányába.

Az MVP-ben **nem kell** implementálni:

- online fizetést,
- automatikus számlázást,
- Pixieset integrációt,
- privát PIN-es ügyfélgalériát,
- rendelési rendszert,
- saját képfeltöltő rendszert.

1. Üzleti cél

A Zsana Photo egy fotós szolgáltatásokat bemutató, modern, mobilbarát weboldal.

A látogató elsődleges célja:

1. megismerni a fotóst és a szolgáltatásokat;
2. megnézni referenciatételeket;
3. időpontot foglalni családi fotózásra;
4. intézményi fotózás esetén kapcsolatfelvételi/ajánlatkérési igényt küldeni.

A fotós elsődleges célja:

1. látni az összes foglalást;
2. kezelni tudja az elérhető időszakokat;

3. eldönthesse, hogy a foglalások automatikusan vagy manuálisan legyenek jóváhagyva;
4. e-mailben értesítést kapjon az új foglalásokról;
5. az ügyfél automatikus e-mailes visszaigazolást kapjon.

2. Technológiai döntés

Kötelező stack

- Next.js - App Router
- TypeScript - strict mode
- React
- Tailwind CSS
- PostgreSQL
- Neon PostgreSQL
- Drizzle ORM
- Zod
- React Hook Form
- Resend e-mail küldéshez
- Vercel deployment

Verziókezelés

A projekt létrehozásakor az adott időpontban elérhető stabil Next.js verziót használd. Ne használj elavult, szükségtelenül fixált verziót csak azért, mert a specifikáció régebbi.

Minimum Node.js: a kiválasztott Next.js stabil verzió által támogatott LTS Node.js verzió.

Fontos architektúráli döntés

Nem készül külön Express/Nest/.NET backend projekt.

A backend logika a Next.js alkalmazásban legyen:

- Server Actions,
- Route Handlers,
- server-side service layer

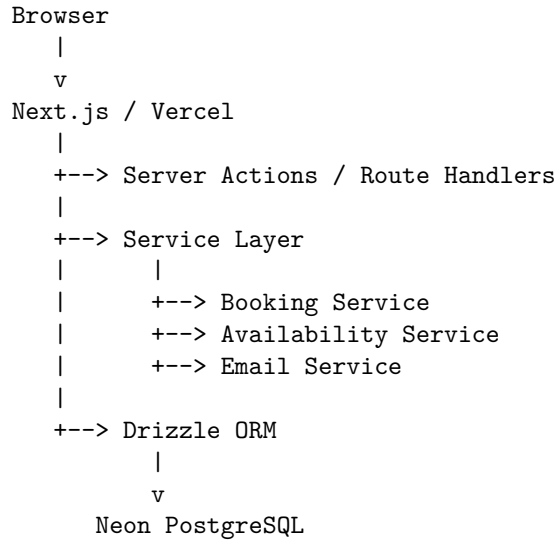
segítségével.

A repository egyetlen alkalmazás:

```
zsanaphoto/  
  app/  
  components/  
  lib/  
  db/
```

```
public/  
...
```

3. Production architektúra



Email:
Next.js -> Resend -> Customer/Admin

3/A. Lokális és Vercel futtatási követelmények

A projektnek **ugyanazzal a forráskóddal** teljes értékűen működni kell:

1. lokális fejlesztői környezetben;
2. Vercelen production környezetben.

A környezetek között ne legyen eltérő üzleti logika. A különbségeket kizárólag environment variable-ok és külső erőforrások konfigurációja adja.

3/A.1. Környezetek

Két adatbázis-környezet legyen:

```
zsanaphoto-dev  
zsanaphoto-production
```

A lokális fejlesztés kizárólag a **zsanaphoto-dev** adatbázist használja. A Vercel Production kizárólag a **zsanaphoto-production** adatbázist használja.

A két adatbázis adatai soha ne keveredjenek.

3/A.2. Environment variables

Kötelezően legyen `.env.example`:

```
DATABASE_URL=
RESEND_API_KEY=
EMAIL_FROM=
ADMIN_NOTIFICATION_EMAIL=
ADMIN_AUTH_SECRET=
NEXT_PUBLIC_APP_URL=http://localhost:3000
```

Lokális környezetben ezek a `.env.local` fájlban legyenek megadva. Vercelen ugyanezeket a Vercel Environment Variables között kell definiálni.

Titkos értékek soha ne kerüljenek Gitbe.

3/A.3. Local development

A projekt rootjában a minimális indítás:

```
pnpm install
pnpm db:migrate
pnpm db:seed
pnpm dev
```

Az alkalmazás:

`http://localhost:3000`

A lokális alkalmazás a dev Neon adatbázishoz csatlakozzon. Külön lokális PostgreSQL telepítése nem szükséges.

3/A.4. Vercel deployment

A repository közvetlenül importálható legyen Vercelbe.

Nem készül:

- külön Node.js server;
- Express/NestJS server;
- külön backend deployment;
- custom server entrypoint.

A Vercel build a Next.js alkalmazást deployolja.

Production environment variable:

```
DATABASE_URL=<production Neon connection string>
NEXT_PUBLIC_APP_URL=https://zsanaphoto.com
```

A development / preview environment használhat külön Neon adatbázist.

3/A.5. Database migration és seed

A repository tartalmazzon működő scriptet:

```
pnpm db:generate
pnpm db:migrate
pnpm db:seed
```

A deployment során a production adatbázis migration kezelését biztonságosan, explicit módon kell megoldani. Ne fusson minden alkalmazásindításkor automatikus destructive migration.

3/A.6. Email lokálisan és productionben

Productionben a Resend legyen használva.

Lokális fejlesztésben két elfogadható mód van:

1. ha a Resend kulcs rendelkezésre áll, valódi teszt e-mail küldhető;
2. ha nincs Resend konfiguráció, `ConsoleEmailProvider` logolja az e-mailek tartalmát a terminálba.

A lokális futásnak Resend API key nélkül is el kell indulnia.

3/A.7. Production adatbiztonság

A production DB connection stringet soha ne hardcode-old.

Frontend bundle-be ne kerüljön:

- DATABASE_URL;
- RESEND_API_KEY;
- ADMIN_AUTH_SECRET;
- egyéb secret.

A database és e-mail hozzáférés kizárólag server-side kódból történhet.

4. Publikus oldalak

Kötelező route-ok

```
/
/csaladi-fotozas
/intezmenyi-fotozas
/galeria
/rolam
/kapcsolat
/idopontfoglalas
```

Opcionális előkészített route-ok

/adatvedelem

/aszf

Ezek tartalma MVP-ben lehet placeholder, de az útvonalak legyenek előkészítve.

5. Főoldal

A főoldal legyen fotóközpontú, prémium és egyszerű.

Kötelező szekciók

1. Hero
2. Családi fotózás
3. Intézményi fotózás
4. Válogatott referenciafotók
5. Rólam rövid blokk
6. Hogyan működik?
7. Időpontfoglalás CTA
8. Kapcsolat
9. Footer

Hero

Cím:

Emlékek, amiket jó újra és újra megnézni.

Alcím:

Családi és intézményi fotózás természetes, időtálló képekkel.

CTA:

- Családi fotózás
- Időpontot foglalok

6. Design rendszer

A kliens kedvenc színe a piros.

A piros legyen accent szín, ne az egész oldal domináns háttérszíne.

Javasolt vizuális irány

- törtfehér / világos háttér
- nagyon sötét szöveg
- piros CTA / accent

- nagy, lélegző fotók
- kevés árnyék
- finom border-ek
- elegáns tipográfia
- minimális animáció

Kerülendő

- túlzottan corporate megjelenés
- túl sok card
- neon színek
- túl sok animáció
- tech-dashboard jellegű publikus oldal

7. Responsive követelmények

A rendszer mobile-first.

Kötelezően használható legyen legalább:

- 360 px
- 390 px
- tablet
- desktop

A foglalás teljes folyamata mobilról is könnyen kitölthető legyen.

Touch targetek legyenek megfelelő méretűek, kb. 44 px körüli minimum inter-aktív célmérettel.

8. Szolgáltatások

Az MVP-ben két alap szolgáltatás legyen.

8.1. Családi fotózás

- valódi online időpontfoglalás
- kiválasztható időpont
- ügyféladatok
- megjegyzés
- automatikus vagy manuális jóváhagyás

8.2. Intézményi fotózás

Az első verzióban ne legyen normál időpontfoglaló flow.

Legyen kapcsolatfelvételi / ajánlatkérési form:

- intézmény neve
- kapcsolattartó neve
- e-mail
- telefon
- körülbelüli létszám
- kívánt időszak
- üzenet

A form submit után demo/sikeres visszajelzés jelenjen meg.

Valódi intézményi booking workflow későbbi fázis.

9. Időpontfoglalás - elsődleges üzleti funkció

Route:

/idopontfoglalas

9.1. Első lépés

A felhasználó válasszon szolgáltatást:

- Családi fotózás
- Intézményi fotózás

A családi fotózás esetén induljon a tényleges slot foglalás.

Az intézményi választás vezessen ajánlatkérési flow-ba.

10. Családi fotózás foglalási flow

1. Szolgáltatás
2. Dátum
3. Elérhető időpont
4. Ügyféladatok
5. Összegzés
6. Foglalás elküldése
7. E-mail
8. Sikeres oldal

11. Foglalható időpontok kezelése

A rendszer ne fix, frontendbe égetett időpontokat használjon.

A szabad időpontokat az adatbázisban tárolt szabályokból kell kiszámítani.

Availability rule

Például:

```
Hétfő  
09:00 - 17:00  
slot: 60 perc  
buffer: 15 perc
```

A rendszer ebből generálja a foglalható kezdési időpontokat.

12. Szolgáltatás beállításai

Minden szolgáltatás rendelkezzen:

```
id  
name  
slug  
description  
durationMinutes  
bufferMinutes  
approvalMode  
active  
sortOrder
```

approvalMode

Két érték:

```
AUTO  
MANUAL
```

Ez kritikus üzleti szabály.

13. Automatikus vs manuális jóváhagyás

Az adminban és szolgáltatásonként legyen állítható.

AUTO

A szabad időpont lefoglalásakor:

```
PENDING -> CONFIRMED
```

Az ügyfél automatikus visszaigazolást kap.

MANUAL

Új foglalás:

PENDING

Az adminnak jóvá kell hagynia.

Admin lehetőségek:

- Confirm
- Cancel

Confirm:

PENDING -> CONFIRMED

Cancel:

PENDING -> CANCELLED

Az ügyfél manuális módban először:

Foglalási igényét rögzítettük. A végleges visszaigazolásról e-mailben értesítjük.

üzenetet kapjon.

14. Ajánlott foglalási státuszok

PENDING

CONFIRMED

CANCELLED

COMPLETED

NO_SHOW

MVP-ben legalább:

- PENDING
- CONFIRMED
- CANCELLED

legyen kezelve.

15. Foglalási adatok

A booking minimum:

id
bookingNumber
serviceId
customerName
customerEmail
customerPhone
startAt

endAt
status
notes
createdAt
updatedAt
confirmedAt
cancelledAt

Időzónát is kezelj.

A projekt üzleti időzónája:

Europe/Budapest

Az adatbázisban ajánlott UTC-ben tárolni az időpontokat, a UI-ban Europe/Budapest szerint megjeleníteni.

16. Dupla foglalás elleni védelem

Ez kötelező.

A dupla foglalás ellen nem elég frontend ellenőrzés.

A szerveroldalon a booking létrehozása során újra ellenőrizni kell, hogy az adott időintervallum nem ütközik aktív foglalással.

Az üzleti szabály:

- CONFIRMED foglalás blokkolja a slotot;
- PENDING foglalás is blokkolja a slotot, amíg admin döntés nem születik;
- CANCELLED foglalás nem blokkolja;
- NO_SHOW és COMPLETED már múltbeli adat, nem akadályoz új bookingot.

A slot ellenőrzést tranzakcionális / concurrency-safe módon kell megvalósítani.

PostgreSQL oldalon használj olyan stratégiát, amely ténylegesen megakadályozza a race conditiont.

Egyszerű megoldásként a booking létrehozása tranzakcióban történjen, megfelelő lock/constraint stratégiával.

Ne bízz a kliens által elküldött szabad slot listában.

17. Foglalási időpontok generálása

A szabad slotokhoz figyelembe kell venni:

1. service duration
2. buffer
3. weekly availability rules

4. blocked periods
5. existing bookings
6. minimum booking lead time
7. maximum advance booking window

MVP alapértelmezett:

```
minimumLeadTimeHours = 2  
maxAdvanceDays = 90
```

Ezek legyenek adminban módosítható beállítások vagy konfigurációk.

18. Availability szabályok

Adminból lehessen kezelni legalább:

- hétfő
- kedd
- szerda
- csütörtök
- péntek
- szombat
- vasárnap

munkanap státuszt és munkaidőt.

Például:

```
Monday: 09:00 - 17:00  
Tuesday: 09:00 - 17:00  
Wednesday: 09:00 - 17:00  
Thursday: 09:00 - 17:00  
Friday: 09:00 - 15:00  
Saturday: inactive  
Sunday: inactive
```

19. Blocked periods

Adminnak legyen lehetősége időszakot blokkolni.

Példák:

- szabadság
- másik fotózás
- betegség
- egyéb elfoglaltság

Mezők:

startAt
endAt
reason

Ha blocked period ütközik egy időablakkal, ne legyen foglalható.

20. Foglalási határidők

Admin beállítható legyen:

Minimum előfoglalási idő

Például:

2 óra

Tehát ha most 10:15 van, a 10:30-as slot már nem választható.

Maximum előrefoglalás

Például:

90 nap

Az ennél későbbi időpontok ne jelenjenek meg.

21. Ügyféloldali booking UI

A foglalási UI legyen wizard jellegű, de egyszerű.

Step 1

Szolgáltatás.

Step 2

Dátumválasztó.

Step 3

Elérhető időpontok.

Step 4

Adatok.

Step 5

Összegzés.

Step 6

Success.

A user mindig lássa, hol tart.

22. Dátumválasztó

Mobilon és desktopon is kényelmes legyen.

Ne engedje kiválasztani:

- múltat,
- tiltott napot,
- elérhetetlen napot.

Az elérhető napok vizuálisan különbözzenek.

23. Időpontok megjelenítése

Például:

10:00
11:15
12:30
13:45
15:00

A slot generátor a duration + buffer szerint számoljon.

24. Ügyféladatok

Minimum:

name - required
email - required
phone - required
notes - optional

Minden form input labelt tartalmazzon.

Zod validáció:

- valid email
- név minimum hossz
- telefonszám nem üres
- notes opcionális

25. Booking összegzés

Foglalás előtt látszódjon:

Családi fotózás
2026. október 12.
14:00 - 15:15

Név: Kovács Anna
E-mail: anna@example.com
Telefon: +36...

[Foglalás véglegesítése]

26. Success oldal

Sikeres foglalás esetén:

Sikeres foglalás

Köszönjük a foglalást!

Foglalási azonosító:
ZS-2026-0012

Időpont:
2026. október 12. 14:00

AUTO módban:

Az időpontod visszaigazolvá.

MANUAL módban:

A foglalási igényedet rögzítettük. A végleges visszaigazolást
e-mailben küldjük.

27. E-mail rendszer

Használj Resendet production e-mailhez.

Legyen provider abstraction.

```
interface EmailProvider {  
    sendBookingCreatedEmail(input: BookingEmailInput): Promise<void>;  
    sendAdminNewBookingEmail(input: BookingEmailInput): Promise<void>;  
    sendBookingConfirmedEmail(input: BookingEmailInput): Promise<void>;  
    sendBookingCancelledEmail(input: BookingEmailInput): Promise<void>;  
}
```

28. E-mail események

Új booking

Adminnak mindig.

Ügyfélnek:

- AUTO esetén visszaigazolásként;
- MANUAL esetén foglalási igényként.

Confirm

Ügyfélnek e-mail.

Cancel

Ügyfélnek e-mail.

29. Ügyfél e-mail tartalom

AUTO

Subject:

Időpontfoglalás visszaigazolása - Zsana Photo

Body:

Kedves {name}!

Sikeresen lefoglaltad az alábbi időpontot:

Fotózás: {serviceName}

Dátum: {date}

Időpont: {startTime} - {endTime}

Foglalási azonosító: {bookingNumber}

Várunk szeretettel!

Zsana Photo

MANUAL

Subject:

Foglalási igény érkezett - Zsana Photo

A tartalom jelezze, hogy az időpont véglegesítése admin jóváhagyásához kötött.

30. Admin e-mail

Subject:

Új foglalás érkezett - {bookingNumber}

Tartalom:

- ügyfél
- szolgáltatás
- időpont
- telefon
- e-mail
- notes
- aktuális státusz

31. Admin felület

Route prefix:

/admin

Az admin production authenticationt igényel.

MVP-ben ne legyen publikus admin adat.

32. Admin oldalak

/admin

/admin/bookings

/admin/services

/admin/availability

/admin/blocked-periods

/admin/settings

33. Admin dashboard

Mutassa:

- mai foglalások
- következő 7 nap foglalásai
- pending foglalások
- confirmed foglalások
- cancellation count

Példa kártyák:

Mai foglalások	3
Függőben	2

Megerősítve	7
Következő 7 nap	12

34. Admin foglalások oldal

Route:

`/admin/bookings`

Desktopon táblázat.

Mobilon kártya/lista nézet.

Oszlopok:

- booking number
- date
- time
- customer
- service
- status
- createdAt
- actions

Szűrések:

- date range
- status
- service

35. Booking detail

Kattintás után detail page vagy drawer.

Mutassa:

- összes booking adat
- customer
- service
- date/time
- notes
- status
- createdAt
- updatedAt

Műveletek:

- confirm
- cancel
- mark completed

Módosítás csak szerveroldali actionön keresztül történhet.

36. Admin szolgáltatások

Route:

`/admin/services`

A fotós lássa és módosíthassa:

- service name
- description
- duration
- buffer
- approval mode
- active

37. Admin availability

Route:

`/admin/availability`

Heti munkaidő-szabályok kezelése.

Például:

Hétfő	09:00 - 17:00	[x]
Kedd	09:00 - 17:00	[x]
Szerda	09:00 - 17:00	[x]
Csütörtök	09:00 - 17:00	[x]
Péntek	09:00 - 15:00	[x]
Szombat		[]
Vasárnap		[]

38. Admin blocked periods

Route:

`/admin/blocked-periods`

CRUD:

- create
- update
- delete

Példa:

2026-11-01 00:00

-

2026-11-08 23:59

Szabadság

39. Admin settings

Minimum:

- timezone
- admin notification email
- minimum lead time
- maximum booking horizon
- site contact email

40. Admin authentication

Production admin auth kötelező.

Az implementáció használjon modern, jól támogatott authentication megoldást.

A konkrét auth package kiválasztásakor az AI az aktuális Next.js kompatibilis, széles körben támogatott megoldást használja.

Elvárás:

- secure session
- HttpOnly cookie
- CSRF védelem, ha releváns
- password hashing, ha password auth készül
- rate limiting / brute-force protection az auth végponton
- admin route protection middleware vagy equivalent server-side guard segítségével.

A publikus user bookinghez **nem szükséges ügyfélfiók**.

41. Adatmodell

Service

id
name
slug
description
durationMinutes
bufferMinutes

approvalMode
active
sortOrder
createdAt
updatedAt

AvailabilityRule

id
dayOfWeek
startTime
endTime
active
createdAt
updatedAt

BlockedPeriod

id
startAt
endAt
reason
createdAt
updatedAt

Booking

id
bookingNumber
serviceId
customerName
customerEmail
customerPhone
startAt
endAt
status
notes
createdAt
updatedAt
confirmedAt
cancelledAt

AdminUser

id
email

passwordHash
name
createdAt
updatedAt

SiteSettings

id
timezone
adminNotificationEmail
minimumLeadTimeHours
maxAdvanceDays
createdAt
updatedAt

42. Booking number

Olvasható booking number kell.

Példa:

ZS-2026-0012

A booking DB primary key-je ettől külön legyen.

43. Database indexes

Minimum indexek:

Booking

- serviceId
- startAt
- endAt
- status
- customerEmail

BlockedPeriod

- startAt
- endAt

AvailabilityRule

- dayOfWeek
- active

A pontos index-stratégiát az implementáció során az AI a query-k alapján finomíthatja.

44. Database migration

Drizzle migrations használata kötelező.

A repository tartalmazza:

```
db/  
  schema.ts  
  migrations/  
  seed.ts
```

A projekt README-jében legyen dokumentálva:

```
pnpm db:generate  
pnpm db:migrate  
pnpm db:seed
```

45. Environment variables

`.env.example` minimum:

```
DATABASE_URL=  
RESEND_API_KEY=  
EMAIL_FROM=  
ADMIN_NOTIFICATION_EMAIL=  
AUTH_SECRET=  
NEXT_PUBLIC_SITE_URL=http://localhost:3000
```

Minden secret csak server-side legyen használható.

Ne használj `NEXT_PUBLIC_` prefixet secret értékekhez.

46. Resend e-mail domain

Productionban a megfelelő küldő domain beállítása szükséges.

Lokálisan, ha nincs Resend konfiguráció, az alkalmazás ne omoljon össze fejlesztés közben.

Javasolt fallback:

```
console mail provider
```

Developmentben az e-mail tartalmát logolja a szervert console-ba, ha `RESEND_API_KEY` nincs beállítva.

Ez legyen explicit dokumentálva.

47. Provider abstraction

Kötelező a service/provider réteg.

EmailProvider

```
interface EmailProvider {  
  sendBookingCreatedEmail(input: BookingEmailInput): Promise<void>;  
  sendAdminNewBookingEmail(input: BookingEmailInput): Promise<void>;  
  sendBookingConfirmedEmail(input: BookingEmailInput): Promise<void>;  
  sendBookingCancelledEmail(input: BookingEmailInput): Promise<void>;  
}
```

A későbbi integrációk miatt a UI ne ismerje a Resend API-t.

48. Booking service

A foglalási üzleti logika központi service-ben legyen.

Például:

```
createBooking()  
getAvailableSlots()  
confirmBooking()  
cancelBooking()  
completeBooking()
```

Ne az egyes React komponensekben legyen üzleti logika.

49. Availability service

Központi service:

```
getAvailableDates()  
getAvailableSlots()  
isSlotAvailable()
```

A service egyesítse:

- availability rules
- blocked periods
- bookings
- lead time
- booking horizon
- service duration
- buffer

információit.

50. Server Actions / Route Handlers

Használható például:

```
createBooking
confirmBooking
cancelBooking
createBlockedPeriod
deleteBlockedPeriod
updateService
updateAvailabilityRule
```

Publikus GET-like lekérésekhez Route Handler vagy Server Component query használható.

Admin mutation legyen server-side.

51. Security requirements

Minimum:

- HTTPS productionban
- secure cookies
- HttpOnly session cookie
- server-side authorization
- Zod input validation
- SQL injection elleni védelem ORM használatával
- rate limiting booking endpointon
- rate limiting admin loginon
- spam protection booking formon
- email validation
- maximum input lengths

52. Spam protection

MVP-ben legalább egyszerű szerveroldali rate limiting legyen.

Opcionálisan honeypot mező a public booking formhoz.

CAPTCHA nem kötelező MVP-ben.

53. Idempotencia

Booking submit esetén védeni kell a dupla kattintást és ismételt POST-ot.

Frontend:

- submit button disable

- loading state

Backend:

- ellenőrizze az aktív booking konfliktust
- ugyanaz az üzleti művelet ne hozzon létre véletlenül többszörös slot-foglalást.

54. E-mail küldés és tranzakció

A foglalás adatbázisba mentése legyen az elsődleges tranzakció.

Az e-mail küldés ne okozza a booking elvesztését.

Példa:

1. booking létrehozása DB-ben
2. commit
3. e-mail küldés

Ha az e-mail sikertelen:

- booking maradjon meg;
- hiba logolódjon;
- később retry mechanizmus beépíthető.

MVP-ben minimum a hibát logold és az admin számára jelezd szerveroldali logban.

55. UI komponensek

Javasolt shared components:

Header
Footer
Hero
SectionHeading
ServiceCard
PhotoGrid
PhotoCard
BookingWizard
DatePicker
SlotPicker
CustomerForm
BookingSummary
BookingSuccess
AdminSidebar
AdminTable
StatusBadge

ConfirmDialog
Toast
LoadingState
EmptyState

UI komponenseket ne duplikáld route-onként.

56. Admin UI design

Az admin legyen egyszerűbb, mint a publikus oldal.

Desktop:

- sidebar
- content area

Mobile:

- top bar / drawer navigation

A státuszok színes badge-ekkel jelenhetnek meg.

57. Publikus SEO

Kötelező:

- title
- description
- OpenGraph
- canonical URL, ahol releváns
- sitemap
- robots

Privát / admin route-ok ne legyenek indexelhetők.

58. Accessibility

Minimum:

- semantic HTML
- labels
- keyboard navigation
- visible focus
- megfelelő color contrast
- alt text
- modal Escape kezelés
- screen reader friendly állapotjelzések

59. Error handling

A user ne lásson stack trace-et.

Példa:

Valami hiba történt. Kérjük, próbáld meg újra.

Booking conflict esetén:

Ezt az időpontot időközben valaki más lefoglalta. Kérjük, válassz másik időpontot.

60. Loading states

Kötelező:

- slot loading
- booking submit loading
- admin mutation loading
- email-related asynchronous states, ahol releváns

61. Empty states

Példák:

Nincs szabad időpont a kiválasztott napon.

Jelenleg nincs foglalás.

Nincs blokkolt időszak.

62. Seed adatok

Seed után legyen legalább:

Service

Családi fotózás

```
duration: 60 min
buffer: 15 min
approvalMode: AUTO
active: true
```

Intézményi fotózás

```
active: true
```

de ez csak inquiry flow-t használja.

Availability

Hétfő-péntek alapértelmezett minta.

Admin user

Development seed létrehozhat admin usert.

A default jelszó csak development célra legyen használható és dokumentálva.
Productionban kötelező új jelszó.

63. Főoldali referencia galéria

A public gallery MVP-ben statikus/demo képekkel működjön.

Legalább 8-12 vizuálisan egységes kép.

A privát ügyfélgaléria nincs része az MVP-nek.

64. Intézményi inquiry

Route ugyanúgy:

`/idopontfoglalas`

de service selection után az intézményi opció formot nyit.

Submit után:

Köszönjük! Megkaptuk az érdeklődésedet, hamarosan felvesszük veled a kapcsolatot.

MVP-ben ezt az adatot külön DB táblába lehet tenni, ajánlott:

```
InstitutionInquiry
id
institutionName
contactName
email
phone
estimatedParticipantCount
preferredPeriod
message
createdAt
status
```

Status:

```
NEW
CONTACTED
CLOSED
```

Adminban ez külön listázható.

65. Admin inquiry oldal

Route:

`/admin/inquiries`

Mutassa:

- intézmény
- kapcsolattartó
- e-mail
- telefon
- időszak
- státusz
- létrehozva

66. Booking business rules összefoglaló

1. Csak aktív service foglalható.
2. Csak availability szabályon belüli időpont foglalható.
3. Blocked period alatt nincs slot.
4. Lead time-on belüli slot nem foglalható.
5. Booking horizonon kívüli slot nem foglalható.
6. CONFIRMED booking blokkolja a slotot.
7. PENDING booking is blokkolja a slotot.
8. CANCELLED booking felszabadítja a slotot.
9. A backend minden bookingnál újra validálja a slotot.
10. AUTO módban booking létrehozásakor CONFIRMED lesz.
11. MANUAL módban booking létrehozásakor PENDING lesz.
12. Confirm után ügyfél e-mail megy.
13. Cancel után ügyfél e-mail megy.
14. Admin minden új bookingról e-mailt kap.

67. Tesztelési stratégia

Minimum unit tesztelendő üzleti logikák:

- slot generation
- overlap detection
- lead time
- max advance
- approval mode
- booking status transition

Integrációs teszt:

- create booking
- confirm booking
- cancel booking

E2E teszt ajánlott a fő booking flow-ra.

68. Acceptance test - customer

1. Nyisd meg /idopontfoglalas.
2. Válaszd a Családi fotózást.
3. Válassz elérhető dátumot.
4. Válassz szabad slotot.
5. Add meg a nevet/e-mailt/telefont.
6. Küldd el.
7. Siker oldalon jelenjen meg a booking number.
8. AUTO módban az állapot CONFIRMED legyen.
9. A slot ne legyen újra választható.
10. E-mail létrejöttön.

69. Acceptance test - manual approval

1. Service approvalMode = MANUAL.
2. Customer booking létrehozása.
3. Státusz PENDING.
4. Admin kapjon notificationt.
5. Admin megnyitja a bookingot.
6. Confirm.
7. Booking = CONFIRMED.
8. Customer confirmation email menjen.

70. Acceptance test - cancel

1. Booking PENDING vagy CONFIRMED.
2. Admin Cancel.
3. Booking = CANCELLED.
4. Customer cancellation email.
5. A slot újra foglalhatóvá válik.

71. Acceptance test - conflict

1. Ugyanazt a slotot két request próbálja lefoglalni.
2. Pontosan egy aktív booking jöhessen létre.

3. A második request kapjon üzleti hibaüzenetet.
4. Ne legyen adatbázis-korrupció.

72. Acceptance test - admin

Admin:

- login
- dashboard
- bookings
- booking detail
- confirm
- cancel
- services
- approval mode change
- availability rules
- blocked periods
- inquiries

működjön.

73. Definition of Done

A projekt elkészülte azt jelenti, hogy:

- Next.js app buildelhető;
- Vercelen deployolható;
- Neon PostgreSQL használható;
- Drizzle migration működik;
- seed működik;
- admin authentication működik;
- booking flow működik;
- availability számítás működik;
- dupla booking védelem működik;
- AUTO/MANUAL mód működik;
- admin booking management működik;
- institution inquiry működik;
- e-mail service működik vagy lokálisan biztonságosan logolható;
- responsive UI működik;
- lint/typecheck/build hibamentes.

74. Lokális fejlesztés

Kötelező README parancsok:


```
pnpm install
pnpm dev

DB setup:

pnpm db:generate
pnpm db:migrate
pnpm db:seed

Quality:

pnpm lint
pnpm typecheck
pnpm test

Production build:

pnpm build
pnpm start
```

75. Vercel deployment

A repository legyen közvetlenül Vercelbe importálható.

Environment variables Vercelben:

- DATABASE_URL
- RESEND_API_KEY
- EMAIL_FROM
- ADMIN_NOTIFICATION_EMAIL
- AUTH_SECRET
- NEXT_PUBLIC_SITE_URL

A DB migration futtatásának deployment stratégiáját a README dokumentálja.

A build ne függjön lokális fájlrendszertől vagy lokális mock adatbázistól.

76. Projekt fájlstruktúra - ajánlott

```
zsanaphoto/
  app/
    (public)/
      page.tsx
      családi-fotozas/page.tsx
      intezmenyi-fotozas/page.tsx
      galeria/page.tsx
      rolam/page.tsx
      kapcsolat/page.tsx
```

```

    idopontfoglalas/page.tsx
  admin/
    page.tsx
    bookings/page.tsx
    services/page.tsx
    availability/page.tsx
    blocked-periods/page.tsx
    inquiries/page.tsx
    settings/page.tsx
  api/
  globals.css
components/
  public/
  booking/
  admin/
  ui/
db/
  schema.ts
  client.ts
  seed.ts
  migrations/
lib/
  services/
    booking-service.ts
    availability-service.ts
    inquiry-service.ts
  providers/
    email/
      types.ts
      resend-email-provider.ts
      console-email-provider.ts
  validation/
  auth/
  utils/
public/
  images/
drizzle.config.ts
.env.example
README.md
package.json

```

A pontos struktúrát az AI optimalizálhatja, de a layer separation maradjon meg.

77. Kódolási irányelvek az AI számára

- TypeScript strict.
- `any` csak kivételes indokkal.
- Domain logic ne legyen React komponensben.
- Server-side ellenőrzés minden mutációnál.
- Input validáció Zoddal.
- DB access külön rétegben.
- Provider abstraction külső szolgáltatásokhoz.
- Ne duplikáld az üzleti szabályokat.
- Ne használj magic stringeket, ha enum/const értelmesebb.
- Reusable komponenseket használj.
- A publikusan elérhető oldalak legyenek SEO-kompatibilisek.
- Admin route-ok legyenek védettek.

78. Mit NE implementáljon az AI ebben a verzióban

Ne implementáljon:

- Stripe
- Barion
- SimplePay
- Billingo
- Számlázz.hu
- Pixieset API
- ügyfél login
- saját privát gallery auth
- photo ordering
- shopping cart
- payment flow
- invoice PDF
- image upload platform
- R2 storage

Ezeket későbbi fázisban lehet hozzáadni.

79. Későbbi bővítési pontok

A jelenlegi architektúra később legyen bővíthető:

Phase 2

- valódi CMS
- blog admin

- fejlettebb tartalomkezelés
- Google Calendar sync

Phase 3

- Pixieset gallery integration
- family-specific private galleries

Phase 4

- products
- online ordering
- payment provider

Phase 5

- invoice provider
- automated billing

A mostani booking modul maradjon változatlanul használható ezek mellett.

80. AI IMPLEMENTÁCIÓS SORREND

Az AI ne egyszerre építsen mindent.

Prompt 1 - projekt bootstrap

Hozd létre a Next.js + TypeScript + Tailwind projektet. Állítsd be a lint/typecheck/test alapokat.

Prompt 2 - database

Hozd létre a Drizzle + PostgreSQL sémát, migrationöket és seedet.

Prompt 3 - domain/services

Implementáld a booking és availability service-eket unit tesztekkel.

Prompt 4 - public UI

Készítsd el a publikus fotós weboldal főoldalát és szolgáltatás oldalait.

Prompt 5 - booking flow

Készítsd el az ügyféloldali időpontfoglalást valós DB használatával.

Prompt 6 - admin

Készítsd el az admin authot és booking managementet.

Prompt 7 - email

Implementáld a Resend email providert és development console fallbacket.

Prompt 8 - security + concurrency

Auditáld a booking conflict, authorization, validation, rate limiting és session kezelést.

Prompt 9 - polish

Responsive, accessibility, SEO, loading, empty states, error handling.

Prompt 10 - verification

Futtasd a teljes test/lint/typecheck/build folyamatot, javíts minden hibát, és ellenőrizd a teljes booking flow-t.

81. AI EXECUTION RULE

Az AI coding agent mindig:

1. olvassa el a teljes specifikációt;
2. nézze meg a meglévő repository állapotát;
3. ne törjön le meglévő működést;
4. kis, ellenőrizhető lépésekben dolgozzon;
5. minden nagyobb lépés után futtasson tesztet/typechecket;
6. ne implementáljon olyan funkciót, amely nincs ebben az MVP-ben;
7. ne használjon mock helyett production külső szolgáltatást csak azért, hogy gyorsabb legyen;
8. a booking üzleti szabályokat centralizált service-ben tartsa;
9. a foglalás concurrency problémáját valódi szerveroldali védelemmel kezelje;
10. végül dokumentálja a telepítést.

82. Végző cél

Az első verzió legfontosabb KPI-ja nem az, hogy hány funkció van benne.

Az a cél, hogy:

Egy valódi érdeklődő telefonról végig tudjon menni a családi fotózás időpontfoglalásán, a fotós pedig az adminban azonnal lássa és kezelni tudja a foglalást.

A felhasználói útvonal:

Főoldal

- > Családi fotózás
- > Időpontfoglalás
- > Dátum
- > Szabad időpont
- > Ügyféladatok
- > Összegzés
- > Foglalás
- > E-mail
- > Success

A fotós útvonala:

Admin

- > Dashboard
- > Foglalások
- > Foglalás részletei
- > Confirm / Cancel
- > Availability
- > Blocked periods
- > Service approval mode

Ez a teljes első MVP.

83. Végző environment acceptance criteria

A projekt akkor kész, ha az alábbi két külön workflow is működik.

Local

```
pnpm install
pnpm db:migrate
pnpm db:seed
pnpm dev
```

Eredmény:

<http://localhost:3000>

A booking létrehozható, adatbázisba menthető, adminból látható és a helyi e-mail provider kezeli az értesítést.

Vercel

GitHub repository -> Vercel import -> environment variables -> build -> production.

A production URL-ről ugyanaz a booking flow működjön, a production Neon DB-be írjon, és az e-mailek Resenden keresztül menjenek.

A kliensoldali és szerveroldali kód ne tartalmazzon környezetfüggő hardcoded URL-eket vagy adatbázis logikát.